

Homework 2-1:

Intro. to Virtual and Augmented Reality

Unity Scene Construction & Transformation Matrices

학번: 2025021826

이름: 정경서

목차

1. 과제 개요
2. Unity Scene Tree 구조
3. 변환 행렬 (Transformation Matrix) 계산
4. 각 변환 행렬 계산 (단계별로)
5. O121 (0,10,0) → Camera 좌표 변환
6. DFS (Depth-First Traversal) 과 렌더링
7. MatrixDisplay 스크립트 - Unity Inspector 검증

1. 과제 개요

Unity에서 아래와 같은 계층 구조 (Scene Tree)를 구성하고, 각 오브젝트 간의 4x4 변환 행렬 (Transformation Matrix)를 직접 계산한다. 이후 Unity Inspector의 값과 비교하고, 특정 점의 좌표 변환을 계산한 뒤, Depth-First Traversal의 렌더링 원리를 설명한다.

과제 관련 Unity 구현은 [GitHub\(https://github.com/KyungseoJung/Unity_Personal_VR2026_HW1-2_360Scene_JKS\)](https://github.com/KyungseoJung/Unity_Personal_VR2026_HW1-2_360Scene_JKS)의 Scene_HW2-1이름의 Scene에서 확인 가능하다.

2. Unity Scene Tree 구조

과제에서 제시해준 예시와 동일하게 부모-자식 계층 구조로 Scene을 구성했다.

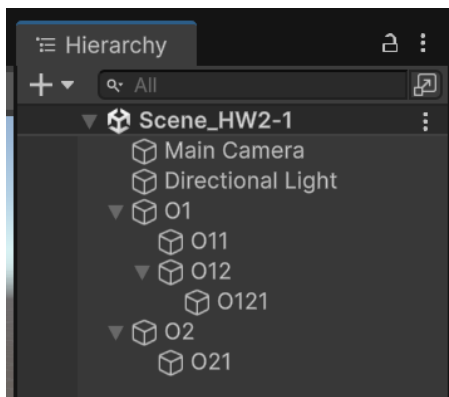
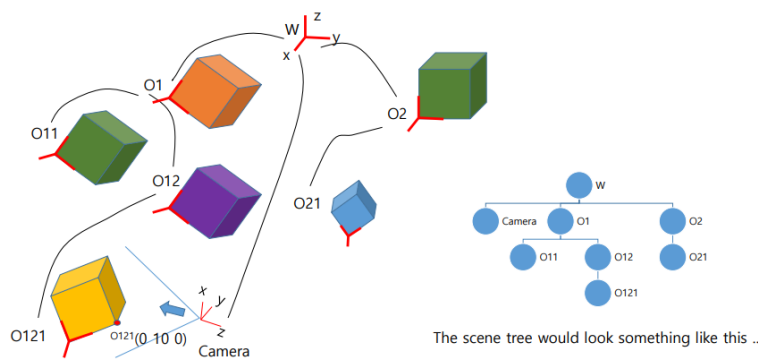


그림 1. Unity scene hierarchy.

World frame W 아래에 Camera, O1, O2가 있고, O1 아래에는 O11, O12이 있고, O12 아래에는 O121이 있고, O2 아래에는 O21이 배치되어 있다.

(W는 World Space를 나타내는 표현이므로, 별도의 오브젝트를 만들지 않았다.)

그리고 각 오브젝트에 입력한 Transform 값은 아래 표와 같다.

Parent	Child	Translation (x,y,z)	Rotation (X,Y,Z)	설명	Unity Hierarchy
W	Camera	(0,0,0)	(90,90,0)	World Space 기준 에서 본 Camera의 좌표(위치/회전)	
W	O1	(20,30,40)	(90,0,90)	World Space 기준 에서 본 O1의 좌표(위치/회 전)	
W	O2	(100,200,0)	(0,90,0)	World Space 기준 에서 본 O2의 좌표(위치/회 전)	
O1	O11	(5,200,300)	(0,0,90)	O1 기준에서 본 O11의 좌표(위치/회 전)	
O1	O12	(30,40,0)	(90,90,90)	O1 기준에서 본 O12의 좌표(위치/회 전)	
O12	O121	(10,20,30)	(90,0,45)	O12 기준에서 본 O121의 좌표(위치/ 회전)	
O2	O21	(40,50,0)	(90,45,0)	O2 기준에서 본 O21의 좌표(위치/회 전)	

3. 변환 행렬 (Transformation Matrix) 계산

- 변환 행렬이란?

각 object의 transform은 translation과 rotation으로 구성된다.

Translation은 object가 부모 좌표계에서 얼마나 이동했는지를 나타내고,

Rotation은 object의 local x, y, z축이 부모 좌표계에서 어떤 방향을 향하는지를 나타낸다.

이 둘을 합치면 아래와 같이 4x4 변환 행렬(Transformation Matrix)가 된다.

즉, 변환 행렬(Transformation Matrix)이란 3D 공간에서 오브젝트의 위치(Translation)와 회전(Rotation)을 하나의 4x4 행렬로 표현한 것이다.

$$T = \begin{bmatrix} R & t \\ 0 & 1 \end{bmatrix} \quad \dots \quad \begin{matrix} R \text{은 } 3 \times 3 \text{ 회전행렬 (어떻게 회전되어 왔는지)} \\ t \text{는 position vector. (원래의 위치 벡터)} \end{matrix}$$

$$T = \begin{bmatrix} r_{11} & r_{12} & r_{13} & t_x \\ r_{21} & r_{22} & r_{23} & t_y \\ r_{31} & r_{32} & r_{33} & t_z \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

● 회전 행렬 계산 원리 - Euler ZXY

과제 표에는 rotation 값이 Euler ZXY형식으로 주어져 있다. 본 계산에서는 표의 rotation 값을 Unity Inspector의 (x,y,z) rotation 값으로 읽고, 과제에서 명시한 Euler ZXY convention에 따라 다음과 같이 rotation matrix를 구성하였다.

0. 회전 행렬 계산

Z축 회전 γ 에 대한 회전: $R_z(\gamma) = \begin{bmatrix} \cos \gamma & -\sin \gamma & 0 \\ \sin \gamma & \cos \gamma & 0 \\ 0 & 0 & 1 \end{bmatrix}$

Y축 회전 β 에 대한 회전: $R_y(\beta) = \begin{bmatrix} \cos \beta & 0 & \sin \beta \\ 0 & 1 & 0 \\ -\sin \beta & 0 & \cos \beta \end{bmatrix}$

X축 회전 α 에 대한 회전: $R_x(\alpha) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos \alpha & -\sin \alpha \\ 0 & \sin \alpha & \cos \alpha \end{bmatrix}$

위의 3개 행렬을 곱하면, 3개 축에 대해 회전이 가능하며, 일반적으로 3개 모두 곱한 것을 3D Rotation Matrix라고 부른다. $\rightarrow$

위 3개 행렬을 곱하면, 3개 축에 대해 회전이 가능하며, 일반적으로 3개 모두 곱한 것을 3D Rotation Matrix라고 부른다.

$$R = R_y R_x R_z$$

(Column vector에 대한 최종 행렬곱 표기는 오른쪽부터 작용하기 때문에, 위와 같은 순서로 표현한다. 실제 곱셈은 오른쪽 행렬부터 곱해서 ZXY 순서가 되는 것)

참고로, 자주 사용하는 삼각함수 값은 아래와 같다.

$$\begin{aligned} \sin(0^\circ) &= 0, & \sin(45^\circ) &= \frac{\sqrt{2}}{2} \text{ (약 0.707)}, & \sin(90^\circ) &= 1 \\ \cos(0^\circ) &= 1, & \cos(45^\circ) &= \frac{\sqrt{2}}{2} \text{ (약 0.707)}, & \cos(90^\circ) &= 0 \end{aligned}$$

4. 각 변환 행렬 계산 (단계별로)

a. $W \rightarrow O1$ (W_T_O1) (${}^W T_{O1}$) | Position: (20,30,40), Rotation: (90,0,90)

① R_Y, R_X, R_Z 를 각각 계산하여, R (회전 행렬)을 얻는다.

(처음 계산에서는 ZYX로 혼동하여, 손으로 필기한 내용 순서가 ZYX이지만, 최종 R 계산은 ZXY 순서로 계산하였다. 이는 4번에서 진행한 모든 손 필기에서 동일하게 적용된다.)

$$\begin{aligned} \text{i) } R_Z(90^\circ) \text{ 계산} &= \begin{bmatrix} \cos 90 & -\sin 90 & 0 \\ \sin 90 & \cos 90 & 0 \\ 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix} \\ \text{ii) } R_Y(0^\circ) \text{ 계산} &= \begin{bmatrix} \cos 0 & 0 & \sin 0 \\ 0 & 1 & 0 \\ -\sin 0 & 0 & \cos 0 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \\ \text{iii) } R_X(90^\circ) \text{ 계산} &= \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos 90 & -\sin 90 \\ 0 & \sin 90 & \cos 90 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 0 & -1 \\ 0 & 1 & 0 \end{bmatrix} \end{aligned}$$

$$R = R_Y R_X R_Z$$

R 은 아래와 같다.

0	-1	0
0	0	-1
1	0	0

② R 에 Translation t 를 합쳐서 최종 변환 행렬(4x4행렬)을 얻는다.

0	-1	0	20
0	0	-1	30
1	0	0	40
0	0	0	1

b. $W \rightarrow O2$ (W_T_O2) (${}^W T_{O2}$) | Position: (100,200,0), Rotation:

(0,90,0)

- ① R_Y, R_X, R_Z 를 각각 계산하여, R (회전 행렬)을 얻는다.

$$\begin{aligned}
 \text{i) } R_Z(0) \text{ 계산} &= \begin{bmatrix} \cos 0 & -\sin 0 & 0 \\ \sin 0 & \cos 0 & 0 \\ 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \rightarrow \text{항등행렬} \\
 \text{ii) } R_Y(90) \text{ 계산} &= \begin{bmatrix} \cos 90 & 0 & \sin 90 \\ 0 & 1 & 0 \\ -\sin 90 & 0 & \cos 90 \end{bmatrix} = \begin{bmatrix} 0 & 0 & 1 \\ 0 & 1 & 0 \\ -1 & 0 & 0 \end{bmatrix} \\
 \text{iii) } R_X(0) \text{ 계산} &= \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos 0 & -\sin 0 \\ 0 & \sin 0 & \cos 0 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \rightarrow \text{항등행렬}
 \end{aligned}$$

$$R = R_Y R_X R_Z$$

R 은 아래와 같다.

0	0	1
0	1	0
-1	0	0

- ② R 에 Translation t 를 합쳐서 최종 변환 행렬(4x4행렬)을 얻는다.

0	0	1	100
0	1	0	200
-1	0	0	0
0	0	0	1

c. $O1 \rightarrow O11$ ($O1_T_O11$) (${}^{O1}T_{O11}$) | Position: (5,200,300), Rotation: (0,0,90)

- ① R_Y, R_X, R_Z 를 각각 계산하여, R (회전 행렬)을 얻는다.

i) $R_z(90^\circ)$ 계산 = $\begin{bmatrix} \cos 90 & -\sin 90 & 0 \\ \sin 90 & \cos 90 & 0 \\ 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix}$

ii) $R_y(0)$ 계산 = $\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \rightarrow$ $\begin{matrix} \text{값들} \\ \text{000} \end{matrix}$

iii) $R_x(0)$ 계산 = $\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \rightarrow$ $\begin{matrix} \text{값들} \\ \text{000} \end{matrix}$

알려진 계산 쪽이
없으면, 주어진 계산 값
선택

$$R = R_y R_x R_z$$

R은 아래와 같다.

0	-1	0
1	0	0
0	0	1

② R에 Translation t를 합쳐서 최종 변환 행렬(4x4행렬)을 얻는다.

0	-1	0	5
1	0	0	200
0	0	1	300
0	0	0	1

d. $O1 \rightarrow O12$ ($O1_T_O12$) (${}^{O1}T_{O12}$) | Position: (30,40,0), Rotation: (90,90,90)

① R_y, R_x, R_z 를 각각 계산하여, R(회전 행렬)을 얻는다.

$$\begin{aligned} \text{i) } R_z(90^\circ) &= \begin{bmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix} \\ \text{ii) } R_y(90^\circ) &= \begin{bmatrix} 0 & 0 & 1 \\ 0 & 1 & 0 \\ -1 & 0 & 0 \end{bmatrix} \\ \text{iii) } R_x(90^\circ) &= \begin{bmatrix} 1 & 0 & 0 \\ 0 & 0 & -1 \\ 0 & 1 & 0 \end{bmatrix} \end{aligned}$$

$$R = R_y R_x R_z$$

R은 아래와 같다.

1	0	0
0	0	-1
0	1	0

② R에 Translation t를 합쳐서 최종 변환 행렬(4x4행렬)을 얻는다.

1	0	0	30
0	0	-1	40
0	1	0	0
0	0	0	1

e. $O12 \rightarrow O121$ ($O12_T_O121$) (${}^{O12}T_{O121}$) | Position: (10,20,30),
Rotation: (90,0,45)

① R_y, R_x, R_z 를 각각 계산하여, R(회전 행렬)을 얻는다.

$$\begin{aligned}
 \text{i) } R_z(45^\circ) \text{ 행렬} &= \begin{bmatrix} \cos 45^\circ & -\sin 45^\circ & 0 \\ \sin 45^\circ & \cos 45^\circ & 0 \\ 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} \frac{\sqrt{2}}{2} & -\frac{\sqrt{2}}{2} & 0 \\ \frac{\sqrt{2}}{2} & \frac{\sqrt{2}}{2} & 0 \\ 0 & 0 & 1 \end{bmatrix} \\
 \text{ii) } R_y(0^\circ) \text{ 행렬} &= \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \quad \rightarrow \text{회전 행렬} \\
 \text{iii) } R_x(90^\circ) \text{ 행렬} &= \begin{bmatrix} 1 & 0 & 0 \\ 0 & 0 & -1 \\ 0 & 1 & 0 \end{bmatrix}
 \end{aligned}$$

$$R = R_y R_x R_z$$

R은 아래와 같다.

$\frac{\sqrt{2}}{2}$	$-\frac{\sqrt{2}}{2}$	0
0	0	-1
$\frac{\sqrt{2}}{2}$	$\frac{\sqrt{2}}{2}$	0

② R에 Translation t를 합쳐서 최종 변환 행렬(4x4행렬)을 얻는다.

$\frac{\sqrt{2}}{2}$	$-\frac{\sqrt{2}}{2}$	0	10
0	0	-1	20
$\frac{\sqrt{2}}{2}$	$\frac{\sqrt{2}}{2}$	0	30
0	0	0	1

f. $O2 \rightarrow O21$ ($O2_T_O21$) (${}^{O2}T_{O21}$) | Position: (40,50,0), Rotation: (90,45,0)

① R_y, R_x, R_z 를 각각 계산하여, R(회전 행렬)을 얻는다.

$$\begin{aligned}
 \text{i) } R_z(0) &= \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \quad \rightarrow \text{항등행렬} \\
 \text{ii) } R_y(45^\circ) &= \begin{bmatrix} \cos 45^\circ & 0 & \sin 45^\circ \\ 0 & 1 & 0 \\ -\sin 45^\circ & 0 & \cos 45^\circ \end{bmatrix} = \begin{bmatrix} \frac{\sqrt{2}}{2} & 0 & \frac{\sqrt{2}}{2} \\ 0 & 1 & 0 \\ -\frac{\sqrt{2}}{2} & 0 & \frac{\sqrt{2}}{2} \end{bmatrix} \\
 \text{iii) } R_x(90^\circ) &= \begin{bmatrix} 1 & 0 & 0 \\ 0 & 0 & -1 \\ 0 & 1 & 0 \end{bmatrix}
 \end{aligned}$$

$$R = R_y R_x R_z$$

R은 아래와 같다.

$\frac{\sqrt{2}}{2}$	$\frac{\sqrt{2}}{2}$	0
0	0	-1
$-\frac{\sqrt{2}}{2}$	$\frac{\sqrt{2}}{2}$	0

② R에 Translation t를 합쳐서 최종 변환 행렬(4x4행렬)을 얻는다.

$\frac{\sqrt{2}}{2}$	$\frac{\sqrt{2}}{2}$	0	40
0	0	-1	50
$-\frac{\sqrt{2}}{2}$	$\frac{\sqrt{2}}{2}$	0	0
0	0	0	1

g. W→Camera (W_T_Cam) (${}^W T_{Cam}$) | Position: (0,0,0), Rotation: (90,90,0)

① R_y , R_x , R_z 를 각각 계산하여, R(회전 행렬)을 얻는다.

$$\begin{aligned}
 \text{i) } R_z(0) &= \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \quad \rightarrow \text{항등행렬} \\
 R_y(90^\circ) &= \begin{bmatrix} 0 & 0 & 1 \\ 0 & 1 & 0 \\ -1 & 0 & 0 \end{bmatrix} \\
 R_x(90^\circ) &= \begin{bmatrix} 1 & 0 & 0 \\ 0 & 0 & -1 \\ 0 & 1 & 0 \end{bmatrix}
 \end{aligned}$$

$$R = R_Y R_X R_Z$$

R은 아래와 같다.

0	1	0
0	0	-1
-1	0	0

② R에 Translation t를 합쳐서 **최종 변환 행렬(4x4행렬)**을 얻는다.

0	1	0	0
0	0	-1	0
-1	0	0	0
0	0	0	1

5. O121 (0,10,0) → Camera 좌표 변환

a. 계산 과정 설명

O121 오브젝트의 로컬 좌표 (0,10,0)을 Camera 기준 좌표로 바꾸려면, 아래처럼 Scene Tree를 거슬러 올라가며 행렬을 차례로 곱해야 한다.

$$\text{Camera 좌표} = \text{inverse_matrix}(W_T_Cam) \times (W_T_O1) \times (O1_T_O12) \times (O12_T_O121) \times [0,10,0,1]^T$$

순서를 정리하면 아래와 같다.

1단계: 점은 O121 로컬 → World 좌표로 변환한다.

$$P_world = (W_T_O1) \times (O1_T_O12) \times (O12_T_O121) \times [0,10,0,1]^T$$

2단계: World → Camera 좌표로 변환한다.

$$P_cam = \text{inverse_matrix}(W_T_Cam) \times P_world$$

b. W_T_Cam 계산

[4. 각 변환 행렬 계산 (단계별로)]의 (g)에서 구한 값을 그대로 이용한다.

0	1	0	0
0	0	-1	0
-1	0	0	0
0	0	0	1

c. O121(0,10,0)을 World 좌표로 변환

아래 행렬들을 순서대로 곱한다.

$$(W_T_O1) \times (O1_T_O12) \times (O12_T_O121)$$

HW2-1의 5번 중 (c)

STEP 1.

$${}^{O12}_T T_{O121} \times [0, 10, 0, 1]^T$$

$$\begin{bmatrix} \frac{\sqrt{2}}{2} & -\frac{\sqrt{2}}{2} & 0 & 10 \\ 0 & 0 & -1 & 20 \\ \frac{\sqrt{2}}{2} & \frac{\sqrt{2}}{2} & 0 & 30 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 0 \\ 10 \\ 0 \\ 1 \end{bmatrix}^T = \begin{bmatrix} -5\sqrt{2}+10 \\ 20 \\ 5\sqrt{2}+30 \\ 1 \end{bmatrix}$$

STEP 2.

$${}^{O1}_T T_{O12} \times [-5\sqrt{2}+10, 20, 5\sqrt{2}+30]^T$$

$$\begin{bmatrix} 1 & 0 & 0 & 30 \\ 0 & 0 & -1 & 40 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} -5\sqrt{2}+10 \\ 20 \\ 5\sqrt{2}+30 \\ 1 \end{bmatrix}^T = \begin{bmatrix} -5\sqrt{2}+40 \\ -5\sqrt{2}+10 \\ 20 \\ 1 \end{bmatrix}$$

STEP 3.

$${}^W_T T_{O1} \times [-5\sqrt{2}+40, -5\sqrt{2}+10, 20, 1]^T$$

$$\begin{bmatrix} 0 & -1 & 0 & 20 \\ 0 & 0 & -1 & 30 \\ 1 & 0 & 0 & 40 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} -5\sqrt{2}+40 \\ -5\sqrt{2}+10 \\ 20 \\ 1 \end{bmatrix}^T = \begin{bmatrix} 5\sqrt{2}+10 \\ 10 \\ -5\sqrt{2}+80 \\ 1 \end{bmatrix}$$

즉, World 좌표 기준 점은 $(5\sqrt{2}+10, 10, -5\sqrt{2}+80, 1)$ 임을 얻을 수 있다.

d. World → Camera 좌표로 변환

inverse_matrix(W_T_Cam)을 구해서 곱해야 한다.

순수 회전행렬의 역행렬은 전치행렬(행과 열을 뒤바꾼 것)과 같다. 따라서 아래와 같이 계산할 수 있다.

<1.5>의 (d)

$$R(W \rightarrow Cam) = \begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & -1 \\ -1 & 0 & 0 \end{bmatrix} \rightarrow R^T = R^T = \begin{bmatrix} 0 & 0 & -1 \\ 1 & 0 & 0 \\ 0 & -1 & 0 \end{bmatrix}$$

$$({}^W T_{Cam})^{-1} = \begin{bmatrix} 0 & 0 & -1 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & -1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

이제 World 좌표를 Camera 좌표로 변환하기 위해 아래와 같이 계산한다.

이제 World 좌표를
Camera 좌표로 변환해라

$$P_W = [5\sqrt{2}+10, 10, -5\sqrt{2}+80, 1]^T$$

$$P_{Cam} = ({}^W T_{Cam})^{-1} \cdot P_W$$

$$= \begin{bmatrix} 0 & 0 & -1 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & -1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 5\sqrt{2}+10 \\ 10 \\ -5\sqrt{2}+80 \\ 1 \end{bmatrix} = \begin{bmatrix} 5\sqrt{2}-80 \\ 5\sqrt{2}+10 \\ -10 \\ 1 \end{bmatrix} \star$$

$\Rightarrow, Cam_{cam} (-72.929, 17.071, -10) \star$



따라서, 0121(0,10,0)를 Camera 좌표로 변환하면 $(-72.929, 17.071, -10)$ 이다.

위 계산 결과는 과제에서 명시한 Euler ZXY 적용 순서를 기준으로 수행하였다.

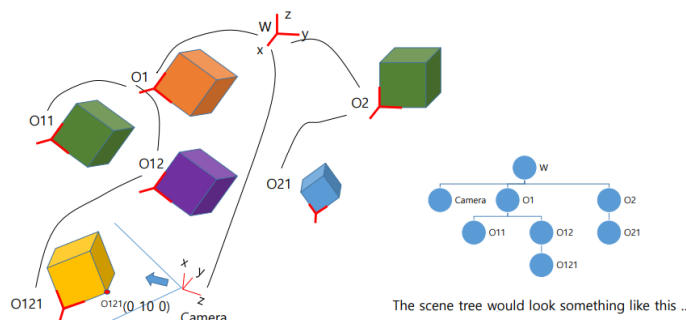
따라서 각 rotation 값 (X, Y, Z)에 대해 $R = R_y \cdot R_x \cdot R_z$ 를 사용하였고, 이 결과는 Unity에서 MatrixDisplay script가 local-to-parent matrix와 일치한다.

이 비교는 [7. MatrixDisplay 스크립트 - Unity Inspector 검증]에서 별도로 비교하였다.

6. DFS (Depth-First Traversal) 과 렌더링

a. Depth-First Traversal 이란?

Depth-First Traversal은 scene tree를 순회할 때 한 branch를 가능한 깊이까지 먼저 내려간 뒤, 더 이상 내려갈 child가 없으면 다시 위로 올라와서 다음 branch를 방문하는 방식이다. 본 과제의 scene tree는 다음과 같은 계층 구조를 가진다.



따라서 가능한 DFS 방문 순서는 다음과 같다.

W → Camera → O1 → O11 → O12 → O121 → O2 → O21

이때 중요한 점은 child object의 transform은 parent object의 transform에 의해 결정된다는 것이다. 예를 들어 O121의 world transform을 구하려면 O121 자신의 local transform만 보면 되는 것이 아니라, parent인 O12, 그 parent인 O1, 그리고 World까지의 transform을 모두 누적해서 곱해야 한다.

즉, $[W_T_O121 = W_T_O1 \times O1_T_O12 \times O12_T_O121]$ 와 같이 parent에서 child 방향으로 transformation matrix를 누적해야 O121의 최종 world position과 orientation을 얻을 수 있다. DFS traversal은 이러한 hierarchical transform을 계산하기에 적합하다. 왜냐하면, scene tree를 내려가면서 현재까지 누적된 transformation matrix를 저장하고, child object를 방문할 때 parent의 누적 matrix에 child의 local matrix를 곱하면 되기 때문이다.

b. 왜 일부 오브젝트만 렌더링되나?

Scene tree 안에 object가 존재한다고 해서 모든 object가 항상 화면에 렌더링되는 것은 아니다. 렌더링 과정에서는 먼저 scene tree를 순회하면서 각 object의 world transform을 계산하고, 그 object가 Camera 기준에서 보이는 위치에 있는지 판단한다.

대표적으로 사용되는 방법이 view frustum culling이다. Camera는 near plane, far plane, field of view에 의해 보이는 공간, 즉 view frustum을 가진다. 어떤 object가 이 view frustum 밖에 있으면, 그 object는 화면에 보이지 않으므로 실제 rendering 대상에서 제외될 수 있다.

또한 object가 비활성화되어 있거나, renderer component가 꺼져 있거나, camera culling mask에 포함되지 않은 layer에 있거나, 다른 object에 의해 완전히 가려지는 등의 경우에도 rendering 대상에서 제외될 수 있다. 따라서 **DFS traversal**은 scene tree의 모든 object를 방문할 수 있지만, 최종적으로 화면에 그려지는 object는 visibility test를 통과한 subset이다.

예를 들어 O2가 camera의 view frustum 밖에 있다면 O2는 rendering되지 않을 수 있

다. O21 역시 O2의 child이므로, O21의 world transform은 O2의 transform을 포함해서 계산된다. 만약 O21도 camera가 보는 영역 밖에 있다면 실제 화면에는 rendering되지 않는다. 이처럼 scene tree traversal은 object들의 계층적 위치와 방향을 계산하는 과정이고, 이를 기반으로 rendering 단계에서는 camera에 보이는 object만 선택되어 화면에 그려진다.

7. MatrixDisplay 스크립트 - Unity Inspector 검증

a. 검증 목적

[4. 각 변환 행렬 계산 (단계별로)]에서 손으로 계산한 변환 행렬(Transformation Matrix)이 Unity에서 실제로 계산되는 matrix와 어떤 관계가 있는지 확인하기 위해, 과제에서 제공된 MatrixDisplay 스크립트를 각 object에 부착하였다. 이 스크립트는 선택한 object의 local-to-parent matrix를 Inspector 창에 출력한다.

여기서 local-to-parent matrix란 child object의 local coordinate를 parent coordinate로 변환하는 matrix이다. 예를 들어, O12에 스크립트를 붙이면 출력되는 matrix는 $O1_T_O12(^{O1}T_{O12})$ 에 해당한다.

b. 손으로 계산한 값(좌)과 Unity 결과 비교(우)

MatrixDisplay script가 출력한 local-to-parent matrix와 손으로 계산한 matrix가 모두 정확히 일치하였다.

1) W_T_O1

#

Matrix Display (Script)

Script

MatrixDisplay

Local to Parent Matrix

0.000-1.0000.00020.000
0.0000.000-1.00030.000
1.0000.0000.00040.000
0.0000.0000.0001.000

0	-1	0	20
0	0	-1	30
1	0	0	40
0	0	0	1

2) W_T_O2

#

✓

Matrix Display (Script)

Script

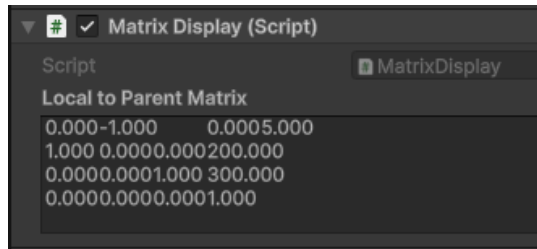
MatrixDisplay

Local to Parent Matrix

0.0000.0001.000 100.000
0.0001.000 0.000200.000
-1.000 0.0000.0000.000
0.0000.0000.0001.000

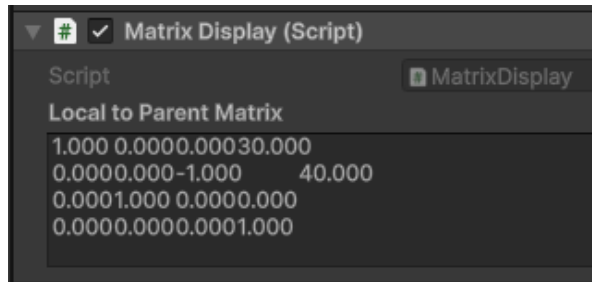
0 ^{e3}	0 ^{e3}	1 ^{e3}	100 ^{e3}
0 ^{e3}	1 ^{e3}	0 ^{e3}	200 ^{e3}
-1 ^{e3}	0 ^{e3}	0 ^{e3}	0 ^{e3}
0 ^{e3}	0 ^{e3}	0 ^{e3}	1 ^{e3}

3) O1_T_O11



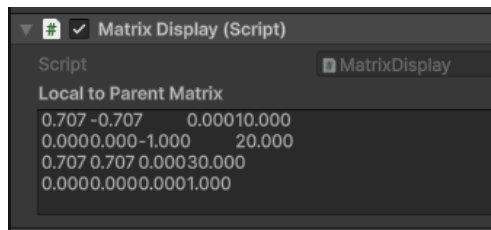
0	-1	0	5
1	0	0	200
0	0	1	300
0	0	0	1

4) O1_T_O12



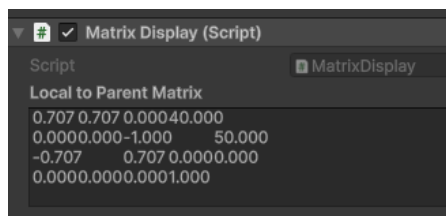
1	0	0	30
0	0	-1	40
0	1	0	0
0	0	0	1

5) O12_T_O121



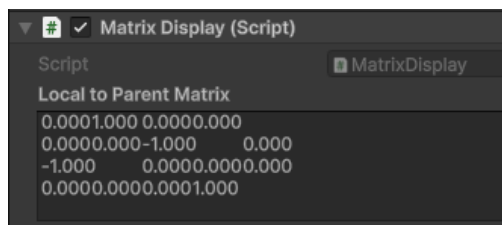
$\frac{\sqrt{2}}{2}$	$-\frac{\sqrt{2}}{2}$	0	10
0	0	-1	20
$\frac{\sqrt{2}}{2}$	$\frac{\sqrt{2}}{2}$	0	30
0	0	0	1

6) O2_T_O21



$\frac{\sqrt{2}}{2}$	$\frac{\sqrt{2}}{2}$	0	40
0	0	-1	50
$-\frac{\sqrt{2}}{2}$	$\frac{\sqrt{2}}{2}$	0	0
0	0	0	1

7) W_T_Cam



0	1	0	0
0	0	-1	0
-1	0	0	0
0	0	0	1